



Procesadores del Lenguaje

Grado en Ingeniería Informática

Segunda entrega práctica final:.

Grupo 81/ Equipo 107

Año académico 2025/2026

Miguel Amiel Gil

| 100525454 | 100525454@alumnos.uc3m.es

Chen Xu

| 100522395 | 100522395@alumnos.uc3m.es

Trabajo realizado durante la segunda sesión

Continuando con el trabajo realizado durante la Semana Santa, hemos implementado las variables locales y el funcionamiento del bucle FOR.

Variables Locales

Para las variables locales, hemos creado la tabla de variables locales que almacena cada uno de los nombres de dichas variables. Para ello, hemos usado un array de strings, un contador de variables y una variable dentro_main que permite saber el scope en el que nos encontramos. Esta última variable es necesaria para poder saber si printear main_variable o variable al utilizar una variable dentro de la función main. Considerando estos añadidos, modificamos nuestro código C de la siguiente forma:

```
char *local_variables[2048]; // Tabla de variables locales
int local_variables_counter = 0; // Contador de variable locales que
tenemos
int dentro_main = 0; // Variable para saber si nos encontramos dentro
del main o no

int es_local(char *var_name){
    for(int i = 0; i < local_variables_counter; i++){
        if(strcmp(var_name,local_variables[i]) == 0){
            // Si coincide el nombre de la variable con alguna del
array, entonces es local.
            return 1;
        }
    }
    // Si no hubo ninguna coincidencia, entonces es global
    return 0;
}

void añadir_variable_local(char *nombre){
    // Añadimos la variable al array y aumentamos el contador
    local_variables[local_variables_counter] = gen_code(nombre);
    local_variables_counter++;
}
```

Es importante destacar las funciones `es_local` y `añadir_variable_local` que permiten comprobar si una variable es local, es decir, se encuentra en la tabla y también añadir una variable a la tabla correspondiente.

Bucle FOR

Después, hemos implementado la estructura de control FOR. Hemos dado por hecho que en la inicialización de la variable, esta será la variable local y no global. Para conservar la lógica anterior de los `setf` y `setq`, hemos primero inicializado la variable antes de entrar al loop for y en esta inicialización se comprueba si la variable ya fue declarada antes. Si la variable ya fue declarada, se usa `setf` y en otro caso sería `setq`. Hemos separado las expresiones en expresiones de comparación y expresiones aritméticas para especificar en la expresión de la estructura de control. Además hemos añadido a FOR, INC y DEC a los tokens.

Por último, otra idea importante es que, durante el desarrollo del bucle FOR, tuvimos que dividir el no terminal "expresion" en 3 no terminales distintos: `expr_condicional`, `expr_others` y término. Esto fue necesario para poder, a través de un único no terminal, referirnos a todas las expresiones condicionales.

Concretamente se modificó el código, generando estas nuevas reglas:

```
sentencia ::= FOR '(' for_var ';' expr_condicional ';' for_operator  
for_var ::= IDENTIF '=' expresion  
expr_condicional -> expresiones condicionales de la gramática original y añadidas es decir,  
operadores( > >= < <= not and...)  
for_operator ::= INC('IDENTIF') ')' while_cont | DEC(' IDENTIF ') ')' while_cont
```

Estructura de control Switch/Case

Para implementar la estructura de control Switch/Case, primero, añadimos dos nuevos tokens SWITCH y CASE para tratar esas palabras clave. Además, añadimos una nueva regla en sentencia para gestionar el SWITCH. Se ha añadido en el no terminal sentencia ya que es este el encargado de generar todas las posibles instrucciones permitidas dentro de una función, entre ellas el switch case.

Concretamente, la regla añadida fue:

```
sentencia ::= SWITCH '(' IDENTIF ')' switch_cont  
  
switch_cont ::= '{' CASE switch_val ':' r_sentencia BREAK ';' switch_cont2 '}'  
switch_val ::= '+' NUMBER %prec UNARY_SIGN |  
              '-' NUMBER %prec UNARY_SIGN |  
              NUMBER  
switch_cont2:  
    lambda |  
    DEFAULT ':' r_sentencia BREAK ';' |  
    CASE switch_val ':' r_sentencia BREAK ';' |
```

Observamos que para la implementación del switch, se han requerido de distintos no terminales auxiliares. Concretamente, switch_cont, switch_val, y switch_cont2.

Switch_cont se utiliza para crear el caso inicial del switch, es decir, especificar el primer case. Al estar lidiando con el caso base, asumimos que al menos se va a requerir de un case dentro del mismo.

A continuación, implementamos switch_cont2 el cual contiene la continuación de dicho switch. Este no terminal permite crear mas cases de forma recursiva y finalizar el switch ya sea añadiendo un default o simplemente lambda(sin default). Para la correcta implementación del switch, se consideró el no terminal switch_val. Este contiene todos los valores posibles que se pueden utilizar en el case del switch. Estos son números con o sin signo.

Funciones

Para la implementación de las funciones, modificamos la gramática añadiendo nuevos no terminales con sus correspondientes normas para poder declararlas correctamente. Para su elaboración, se ha asumido que estas funciones se declaran antes de la función main. Esto es coherente con la idea de que una función debe estar declarada(definida para nuestro lab) para poder ser usada en las funciones localizadas en partes inferiores. Por lo tanto, uno de los primeros cambios realizados fue la adición de una nueva regla en axioma para poder generar las funciones de forma recursiva:

axioma ::= dec_func '}' r_axioma

De esta forma, se permite crear funciones de forma recursiva hasta que se llame a la función main. A continuación, desarrollamos el no terminal llamado dec_func, que contiene la estructura general de la definición de la función:

dec_func ::= IDENTIF '(' func_params_declaration ')' '{' r_sentencia func_return_al_final

func_params_declaration ::= lambda | func_params_types IDENTIF func_declaration_cont

func_params_declaration_cont ::= =lambda | ',' func_params_types IDENTIF func_params_declaration_cont

func_params_types ::= INTEGER | FLOAT | CHAR

func_return_al_final ::= lambda | RETURN expresion ';'

sentencia: func_call

func_params_declaration contiene la declaración de los parámetros posibles para una función. Estos son, lambda, es decir, que no tenga parámetros o de nuevo, un tipo de dato y la variable correspondiente. De esta manera, se permite añadir parámetros de forma recursiva hasta que se desee.

Como no estábamos seguros de que tipos de datos se permiten como parámetros para las funciones, hemos considerado tres tipos básicos de parámetros: int, float y char. Para ello, hemos definido dos nuevos tokens FLOAT y CHAR y hemos reutilizado el token INTEGER. Estos tipos se encuentran recogidos en el no terminal func_params_types.

Además, se puede observar como en sentencia se ha añadido RETURN expresion',' para gestionar el posible return en cualquier lugar de la función. Esto se ha combinado con la regla func_return_al_final, anteriormente mencionada, para poder considerar también ese caso. **Sin embargo, esto nos genera un conflicto reduce/reduce que, hasta el momento, no hemos sido capaces de resolver.**

Llamadas a funciones como procedimientos

Para realizar llamadas a funciones, se ha añadido una nueva regla en sentencia que utiliza un no terminal llamado func_call que recoge la sintaxis necesaria. De esta manera, podemos llamar a funciones en medio de una función, como si fueran una instrucción más. Sin embargo, también es necesario que se pueda usar una llamada a una función como expresión en ifs, printf's y otras funciones. Por lo tanto, también se añadió una regla en expresion que genera func_call para lograr este fin. En resumen, se tienen las siguientes reglas:

expresion ::= func_call

sentencia ::= func_call

func_call ::= IDENTIF '(' func_params_call ')'

func_params_call ::= lambda | expresion func_params_call_cont

func_call permite llamar a una función. Actualmente, hay un problema con este código que no sabemos si tiene que ser implementado, se pueden llamar a funciones no declaradas. Esto será preguntado en la próxima sesión. Con el código actual, además, no se controla que el número de parámetros sea el mismo que los de la función que se está llamando. De esta forma, se permite la adición de parámetros de forma recursiva e indeterminada.

Implementación de arrays

Para crear arrays como variables globales o locales, es decir declararlos, se ha añadido a continue_id la regla:

continue_id ::= '[' expresion ']' continue_comma

Con ello, hemos añadido a las posibles declaraciones anteriores también la declaración del array usando expresiones. De la misma manera, se ha añadido a sentencia una regla idéntica a la que fue definida anteriormente para la asignación de variables. En este caso, el parser utiliza el token del lookahead para diferenciar entre las dos reglas:

sentencia ::= IDENTIF '=' expresion ';'

sentencia ::= IDENTIF '[' NUMBER ']' '=' expresion ';'

Esto nos permite diferenciar entre una asignación de una variable y de un array. Tras esto, necesitamos poder reconocer expresiones que utilicen los ítems de dentro de un array. Por ello, se han añadido

nuevas reglas que permiten reconocer expresiones en las que se usen ítems del array mediante índices. Esto dio lugar a las siguientes reglas:

continue_ID ::= '[' expresion ']' continue_comma

array_index ::= IDENTIF '[' expresion ']'